Near-Real-Time Data Warehouse for Walmart

Course: Data Warehousing & Business Intelligence

By: Muhammad Muneeb Lone (23i-2623)

1. Project Overview

The goal of this project was to create a prototype near-real-time data warehouse for Walmart.

Raw consumer transactions are continuously fed into the system. The ETL pipeline is essential since these incoming records are not complete on their own. Each transaction is "enriched" during transformation by being joined with customer and product master files, adding the product and demographic characteristics needed for insightful analysis.

A multi-threaded Python program is the system's central component. While a second thread uses the HYBRIDJOIN method to combine this stream with the bigger master datasets kept on disk, the first thread simulates a quick and irregular flow of incoming sales transactions. Near-real-time analysis is made possible by the nearly instantaneous loading of the combined, enriched records into the data warehouse.

2. Data Warehouse (DW) Schema

A star schema with numerous dimensions was created and put into use to handle the intricate analytical queries needed for the project, which include analysis by store, supplier, and product.

Five different dimension tables, including DimDate, DimCustomer, DimProduct, DimStore, and DimSupplier, are joined to the core fact table, FactSales, in this structure.

Schema Diagram:

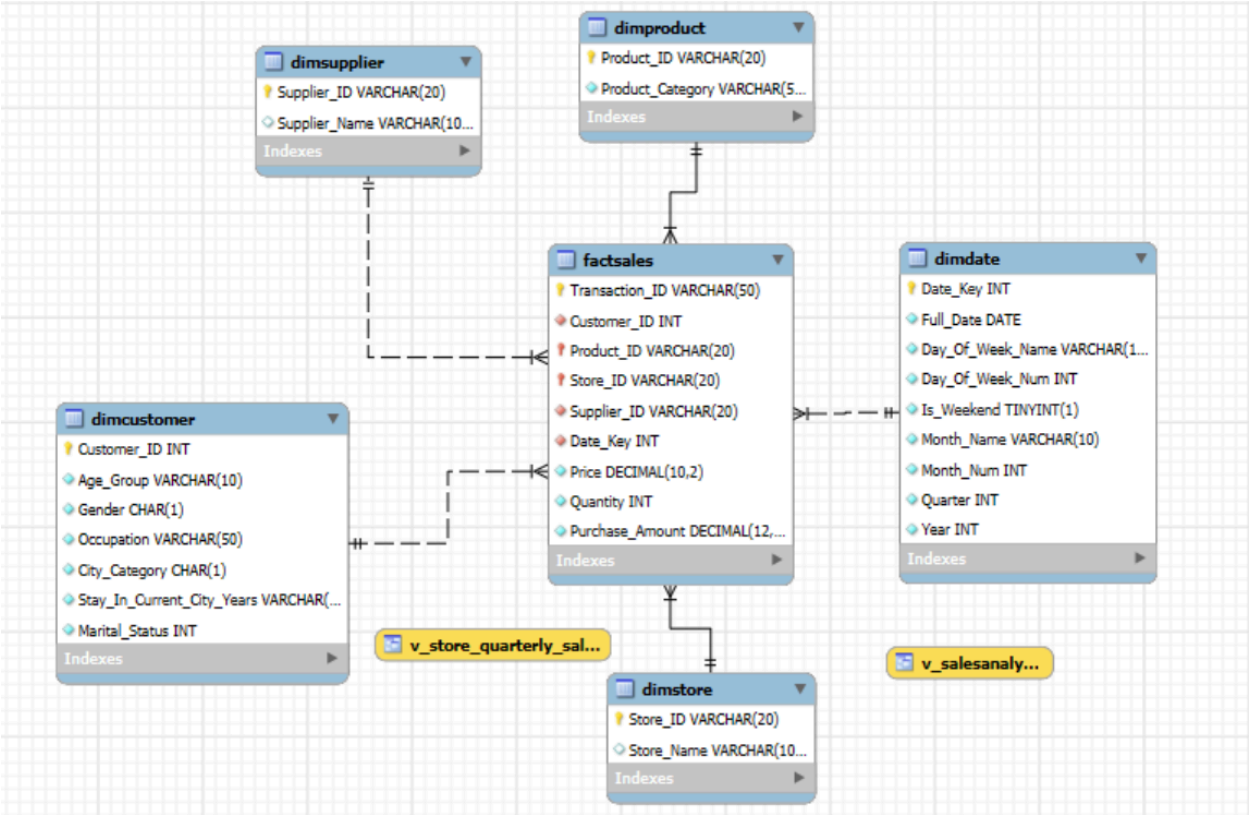


Table Descriptions:

- **FactSales (Fact Table):** This is the central table, with a grain of "one row per product line item on a transaction." It stores all the quantitative, measurable facts of a sale (like Quantity, Price, and Purchase_Amount) and contains foreign keys linking to all five dimension tables.
- **DimDate (Dimension):** A pre-populated table that breaks down a given date into its analytical components (e.g., Month_Name, Quarter, Is_Weekend).
- **DimCustomer (Dimension):** Stores all demographic attributes of a customer, such as Age_Group, Gender, Occupation, and City_Category.
- **DimProduct (Dimension):** Stores attributes specific only to the product, such as its Product_Category.

- **DimStore (Dimension):** A new dimension populated from the master data, holding the unique Store_ID and Store_Name.
- **DimSupplier (Dimension):** A new dimension populated from the master data, holding the unique Supplier_ID and Supplier_Name.

3. The HYBRIDJOIN Algorithm

The implementation follows the stream-cached (S-Cached) variant of the HYBRIDJOIN algorithm described in the project specification. The goal of this algorithm is to efficiently join a fast-arriving stream of records (S) with a large indexed dataset stored on disk (R).

The core logic of this version is to load the stream **(S)** into an in-memory hash table and then use partitions of the relation **(R)** to probe that hash table.

Core Components (as implemented):

1. **Stream Buffer (queue.Queue):** A thread-safe queue that temporarily holds incoming stream tuples.
2. **Hash Table (H) (defaultdict):** An in-memory multi-map (10,000 slots) storing stream tuples keyed by the join attribute.
3. **Queue (Q) (DoublyLinkedList):** A custom thread-safe doubly linked list used for FIFO ordering and O(1) tuple deletion.
4. **Index (JSON File):** A JSON mapping from join keys to the specific disk partition where matching master data resides.
5. **Disk Buffer (Python dict):** A temporary in-memory dictionary that holds one partition (500 tuples) of the master dataset at a time.

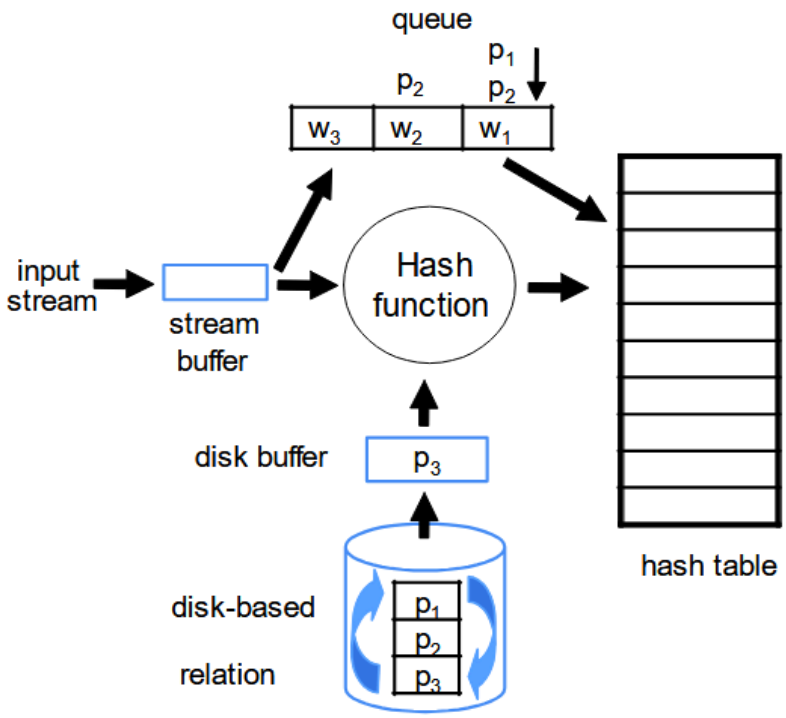
Working Principle:

The join process (hybrid_join_task) runs in an "endless outer loop" as a two-stage pipeline:

1. **Fill the Cache (S):** The algorithm first loads stream tuples from the stream_buffer into its internal Hash Table (H) and Queue (Q), up to the number of available slots (w).

2. **Get Oldest Key (S):** It **peeks** at the oldest tuple at the front of the Queue (Q) to get its join key (e.g., Customer_ID "100123").
3. **Load Partition (R):** It uses this key to look up the correct partition file in the JSON index, then loads that 500-tuple partition (e.g., part_5.csv) into the Disk Buffer.
4. **Probe (R ⋈ S):** It then runs an **inner loop**, iterating through each tuple r in the newly loaded Disk Buffer (R-partition). For each r, it probes the Hash Table (H).
5. **Join and Delete (Output):** If a match is found:
    - The joined output tuple is generated (e.g., (s_tuple, r_tuple, header_map)).
    - The matching stream tuple s is **deleted** from both the Hash Table (H) and the Queue (Q).
    - The available slot counter w is incremented.
6. **Pipeline:** This process completes for Stage 1 (Customer Join), and its output (s_tuple, r_cust_tuple, ...) becomes the input stream for Stage 2 (Product Join), where the entire logic is repeated.

Diagram of HYBRIDJOIN Data Structures:



4. Three Shortcomings of HYBRIDJOIN

The S-Cached HYBRIDJOIN method as described has some serious performance issues despite its effectiveness:

1. **Disk Thrashing:** A poorly ordered stream might result in severe disk thrashing because the algorithm always processes the oldest key first. The technique repeatedly loads and unloads the same files if keys bounce across far-off partitions, which destroys I/O efficiency.
2. **Head-of-Line (HOL) Blocking:** The oldest tuple cannot be avoided by the workflow. Even though thousands of other tuples may be joined instantly at that point, the pipeline as a whole blocks until that particular partition is loaded and processed if that tuple ends up mapping to an infrequently used partition.
3. **Weak Caching Under Skew:** The stream-cached approach becomes ineffective for "hot" keys that occur frequently. The technique discards the partition and reloads it the next time the same key appears, even after merging thousands of identical keys.

5. Reflection on Learnings

This project provided three critical insights:

1. **The Basis is Data Modeling:** Store and supplier data could not be properly handled by the original schema design. The significance of the data model for ETL and analysis was demonstrated by fixing the schema and rebuilding the dimension-population script.
2. **Importance of a Pipeline:** HybridJoin on its own isn’t enough to make the system operate. It doesn't work until the dimension tables are entirely loaded and the R-relation index is ready. From cleaning the data to indexing it to processing it, each step depends on the one before it and proceeds in a logical order.
3. **Algorithms are Constrained by I/O and Data Structures:** The algorithm's performance guarantees had to be met by the doubly linked list. Finding the disk thrashing problem demonstrated how physical I/O patterns and data arrival order may take control of even well-designed algorithms.

6. References

- Naeem, M. A., Dobbie, G., & Weber, G. (2011). *HYBRIDJOIN for Near-Real-Time Data Warehousing*. ICPADS 2011.
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley.
- MySQL. (2025). *MySQL 8.0 Reference Manual*. Oracle.
- Python Software Foundation. (2025). *Python Language Reference*.